

Executive Summary

High-Level Architecture Overview

Layer 1: Load Balancer (Nginx)

Definition

Role in This Architecture

Configuration Details

How It Works

Why Nginx (Not AWS ALB or HAProxy)

Layer 2: Gateway Layer

2.1 API Gateway (Kong / Spring Cloud Gateway)

Definition

Role in This Architecture

How Authentication Works

Rate Limiting Strategy

Circuit Breaker Pattern (Resilience4j)

2.2 WebSocket Gateway (Node.js Cluster)

Definition

Role in This Architecture

How It Works

Room-Based Subscription Model

Why Node.js (Not Java/Spring)

2.3 Grafana Dashboard

Definition

Role in This Architecture

Dashboard Panels

Layer 3: Kafka Cluster (Event Streaming Backbone)

Definition

Why Kafka (Not RabbitMQ, Redis Streams, or AWS SQS)

Kafka Topics in This System

Topic 1: raw-metrics

Topic 2: enriched-metrics

Topic 3: anomaly-events

Partitioning Strategy

Why Partition?

Replication (Fault Tolerance)

Layer 4: Ingestion Cluster

Definition

Why a Separate Ingestion Layer?

Three Core Functions

4.1 Schema Validation

4.2 Metadata Enrichment

4.3 Deduplication (Bloom Filter)

Layer 5: ML Cluster (Anomaly Detection Engine)

Definition

Why Ensemble (Not a Single Model)?

The Three Models

5.1 Model A: Isolation Forest

5.2 Model B: Autoencoder (Neural Network)

5.3 Model C: LSTM (Long Short-Term Memory)

Ensemble Voting (Majority Wins)

Auto-Scaling

Layer 6: Data Layer

6.1 PostgreSQL (Relational Data)

What it stores

Why PostgreSQL (Not MySQL or MongoDB)

6.2 InfluxDB (Time-Series Data)

Definition

What it stores

Why InfluxDB (Not PostgreSQL for metrics)

Retention Policies

6.3 Redis Cluster

Definition

Three Roles in This Architecture

Redis Cluster Topology

Layer 7: Observability Stack

Definition

7.1 Prometheus (Metrics)

What it monitors

How Prometheus Works

7.2 Jaeger (Distributed Tracing)

Definition

Example Trace

Why Tracing Matters

7.3 ELK Stack (Centralized Logging)

Definition

How It Works

Log Format (Structured JSON)

Why Centralized Logging

Complete End-to-End Data Flow

The Journey of a Single Metric Event

Step 1: Data Generation

Step 2: Kafka Receives

Step 3: Ingestion Consumes

Step 4: ML Inference

[Step 5: Backend Processes](#)
[Step 6: Real-Time Broadcast](#)
[Step 7: Alert Fires](#)
[Step 8: Observability Records Everything](#)
[Total End-to-End Latency](#)
[Technology Stack Summary](#)
[Deployment Architecture \(Kubernetes\)](#)
[Interview Talking Points](#)
[Scalability Decisions](#)
[Reliability Decisions](#)
[Data Decisions](#)
[Trade-offs You Should Discuss](#)
[Conclusion](#)

Executive Summary

This document describes the **Architecture 4 — Full Distributed (FAANG-Scale)** design for a Real-Time Anomaly Detection Platform. This architecture is designed to handle **1 million+ events per second**, provide **sub-second anomaly detection**, and demonstrate the kind of system design thinking expected at Google, Meta, Amazon (L4+), and other top-tier tech companies.

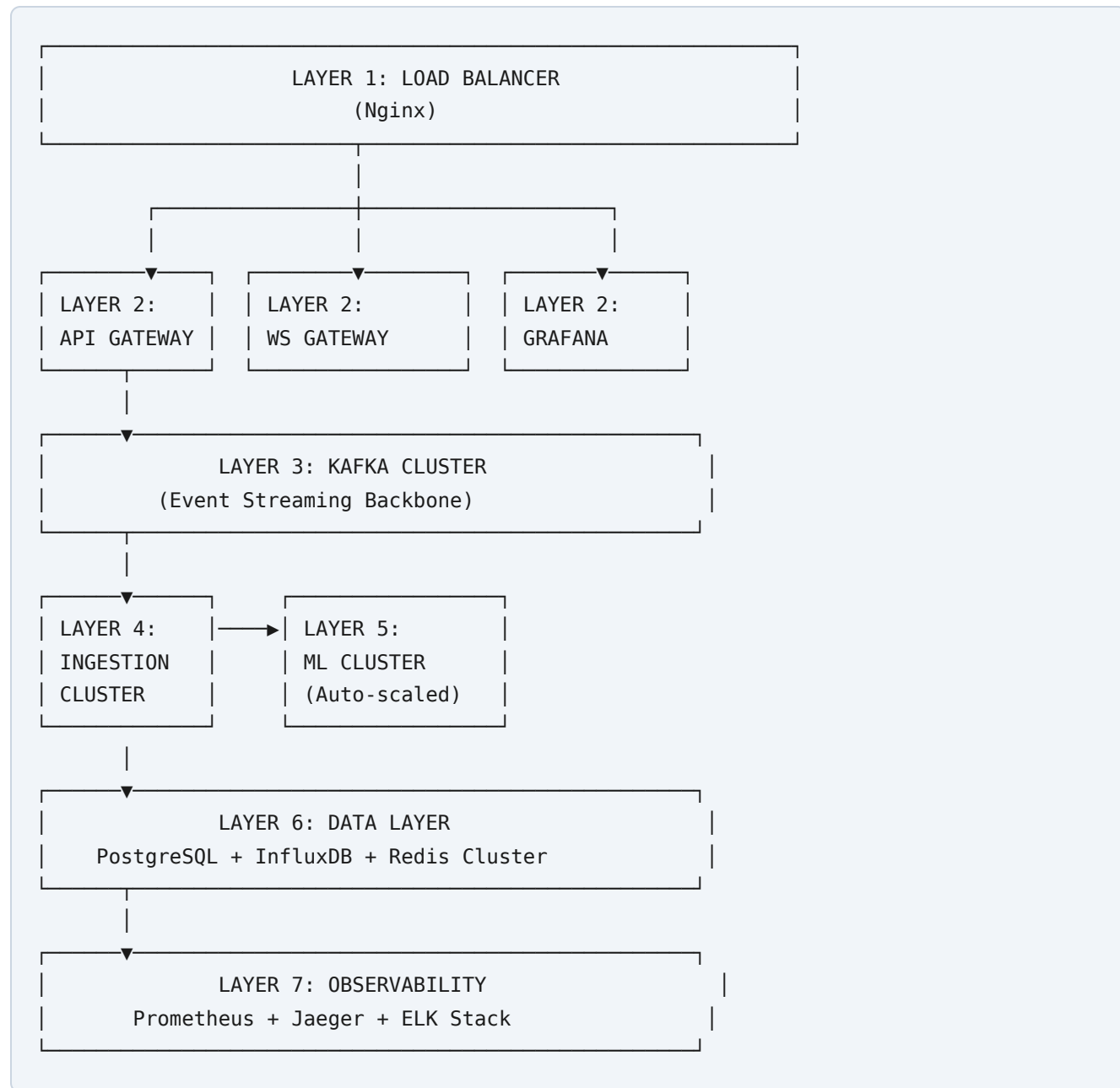
The platform ingests streaming metrics from thousands of data sources, processes them through a distributed pipeline, runs ML-based anomaly detection using an ensemble of models, and delivers real-time alerts and visualizations.

Key Capabilities:

- Ingest 1M+ events/second via Kafka partitioning
 - Sub-second anomaly detection with ML ensemble (Isolation Forest + Autoencoder + LSTM)
 - Real-time dashboard with 100K+ concurrent WebSocket connections
 - Auto-scaling ML inference cluster
 - Full observability with distributed tracing, metrics, and centralized logging
 - Alert deduplication and multi-channel notification (Slack, Email, SMS, Webhook)
-

High-Level Architecture Overview

The system is composed of **7 major layers**, each independently scalable:



Layer 1: Load Balancer (Nginx)

Definition

A **Load Balancer** is a system that distributes incoming network traffic across multiple backend servers to ensure no single server is overwhelmed. It acts as the single entry point for all external traffic.

Role in This Architecture

Nginx sits at the top of the architecture and handles:

- **Traffic Distribution:** Round-robin or least-connections routing to API Gateway instances
- **SSL/TLS Termination:** Handles HTTPS encryption/decryption so backend services don't need to
- **Health Checks:** Monitors backend services and routes traffic away from unhealthy instances
- **WebSocket Upgrade:** Upgrades HTTP connections to WebSocket for the real-time dashboard
- **Rate Limiting (L1):** First layer of defense against DDoS attacks

Configuration Details

Setting	Value	Purpose
Algorithm	Least Connections	Routes to server with fewest active connections
Health Check Interval	5 seconds	Detects failed backends quickly
Max Connections	65,536 per worker	Handles high concurrency
Worker Processes	Auto (1 per CPU core)	Maximizes throughput
Keepalive Timeout	65 seconds	Reuses TCP connections

How It Works

1. Client sends HTTPS request to `api.anomaly-platform.com`
2. DNS resolves to Nginx Load Balancer IP
3. Nginx terminates SSL, inspects the request path
4. Routes to the correct backend:

- `/api/*` → API Gateway cluster
- `/ws/*` → WebSocket Gateway cluster
- `/grafana/*` → Grafana Dashboard

5. If a backend is down, Nginx automatically removes it from the pool

Why Nginx (Not AWS ALB or HAProxy)

- Open source, battle-tested at Netflix, Dropbox, Airbnb
 - Handles 10M+ concurrent connections
 - Acts as both load balancer and reverse proxy
 - Native WebSocket support
 - Can be replaced with AWS ALB in cloud deployments
-

Layer 2: Gateway Layer

2.1 API Gateway (Kong / Spring Cloud Gateway)

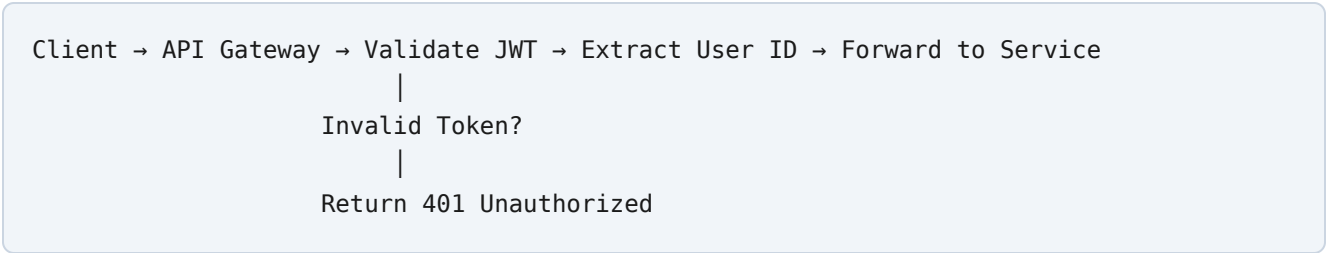
Definition

An **API Gateway** is a server that acts as the single entry point for all API requests. It handles cross-cutting concerns like authentication, rate limiting, request routing, and protocol translation.

Role in This Architecture

Responsibility	Description
Authentication	Validates JWT tokens on every request
Rate Limiting	Limits requests per user/IP (e.g., 1000 req/min)
Throttling	Queues excess requests instead of rejecting them
Request Routing	Routes to correct microservice based on URL path
API Versioning	Supports /v1/, /v2/ endpoints
Request/Response Transform	Adds headers, transforms payloads
Circuit Breaking	Stops sending to a failing service (Resilience4j)

How Authentication Works



1. Client includes Authorization: Bearer <JWT> header
2. Gateway validates token signature using public key (RS256)
3. Extracts user claims (ID, role, permissions)
4. Attaches user context to the forwarded request

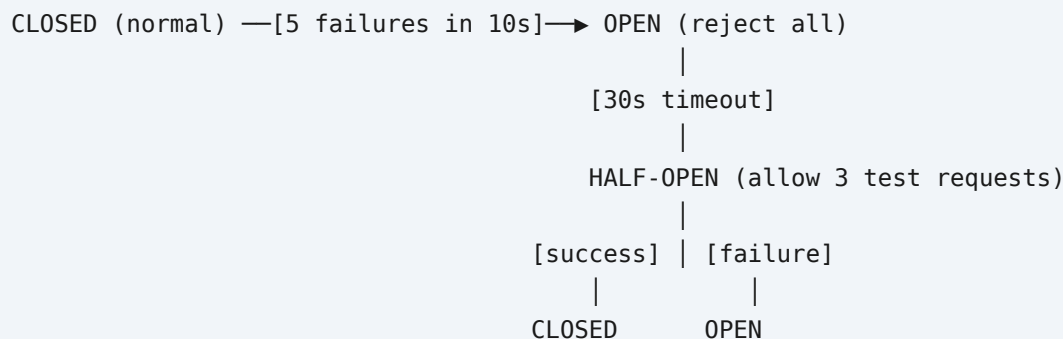
5. Backend services trust the gateway — no re-validation needed

Rate Limiting Strategy

Tier	Limit	Window	Action on Exceed
Free	100 req/min	Sliding window	429 Too Many Requests
Pro	1,000 req/min	Sliding window	429 + Retry-After header
Enterprise	10,000 req/min	Sliding window	Throttle (queue)
Internal Services	Unlimited	N/A	N/A

Rate limiting is implemented using **Redis** with a sliding window counter algorithm.

Circuit Breaker Pattern (Resilience4j)



- **CLOSED:** All requests pass through normally
- **OPEN:** All requests immediately rejected (fail fast) — prevents cascade failure
- **HALF-OPEN:** Allows limited test requests to check if service recovered

2.2 WebSocket Gateway (Node.js Cluster)

Definition

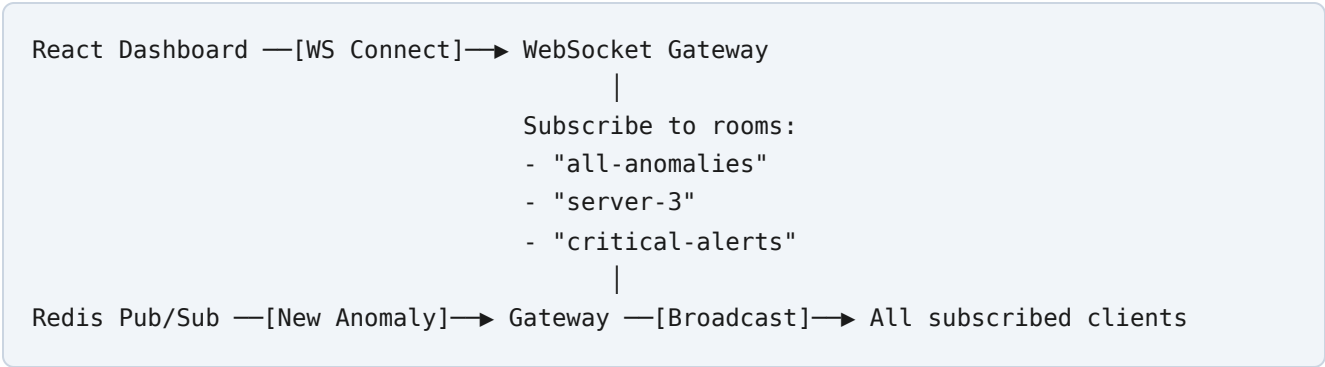
A **WebSocket Gateway** maintains persistent, bidirectional connections with clients for real-time data streaming. Unlike HTTP (request-response), WebSocket keeps the connection open for continuous data flow.

Role in This Architecture

- Maintains **100,000+ concurrent connections** using Node.js event loop

- Implements **room-based subscriptions** (e.g., subscribe to server-3 anomalies only)
- Broadcasts anomaly events from Redis Pub/Sub to connected clients
- Handles connection heartbeats to detect dead clients

How It Works



Room-Based Subscription Model

Room Name	Who Subscribes	What They Receive
all-anomalies	Main dashboard	Every anomaly event
server:{id}	Server detail page	Anomalies for specific server
critical	On-call engineers	Only severity=CRITICAL events
team:{name}	Team dashboards	Anomalies for team's servers

Why Node.js (Not Java/Spring)

- Node.js handles 100K+ concurrent WebSocket connections on a single instance
- Non-blocking I/O is perfect for connection-heavy, low-CPU workloads
- Java WebSocket (Spring) is heavier per connection (~2MB vs ~40KB)
- Node.js Cluster module uses all CPU cores

2.3 Grafana Dashboard

Definition

Grafana is an open-source analytics and monitoring platform that creates real-time dashboards from time-series data sources like InfluxDB and Prometheus.

Role in This Architecture

- Pre-built dashboards for system health monitoring
- Queries InfluxDB for historical time-series metrics
- Queries Prometheus for system-level metrics (CPU, memory of our services)
- Configurable alert thresholds with visual indicators

Dashboard Panels

Panel	Data Source	Shows
Anomaly Rate (live)	InfluxDB	Anomalies detected per minute
Model Accuracy	PostgreSQL	Precision, Recall, F1 over time
Kafka Lag	Prometheus	Consumer group lag per partition
Service Health	Prometheus	CPU, Memory, Error rate per service
Alert Timeline	InfluxDB	Timeline of alerts with severity
Top Anomalous Sources	InfluxDB	Which servers trigger most anomalies

Layer 3: Kafka Cluster (Event Streaming Backbone)

Definition

Apache Kafka is a distributed event streaming platform capable of handling trillions of events per day. It acts as the central nervous system of the architecture — every piece of data flows through Kafka topics.

Why Kafka (Not RabbitMQ, Redis Streams, or AWS SQS)

Feature	Kafka	RabbitMQ	Redis Streams	AWS SQS
Throughput	1M+ msg/sec	50K msg/sec	100K msg/sec	3K msg/sec
Message Retention	Days/Weeks	Until consumed	Configurable	14 days
Replay	Yes (offset reset)	No	Yes	No
Partitioning	Native	Manual	Limited	FIFO only
Ordering	Per partition	Per queue	Per stream	FIFO groups
Consumer Groups	Yes	Yes	Yes	No

Kafka Topics in This System

Topic 1: raw-metrics

Property	Value
Purpose	Raw metric events from all data sources
Partitions	12 (partitioned by source_id)

Property	Value
Replication Factor	3 (across 3 brokers)
Retention	7 days
Producer	Data sources / Collection agents
Consumer	Ingestion Cluster

Message Schema:

```
{
  "event_id": "uuid-v4",
  "source_id": "server-042",
  "source_type": "compute_instance",
  "timestamp": "2026-09-29T14:30:00.123Z",
  "metrics": {
    "cpu_percent": 87.5,
    "memory_percent": 72.3,
    "disk_io_mbps": 450.2,
    "network_in_mbps": 120.5,
    "network_out_mbps": 85.3,
    "request_count": 15420,
    "error_rate": 0.023,
    "latency_p99_ms": 245.7
  },
  "tags": {
    "region": "us-east-1",
    "team": "payments",
    "environment": "production"
  }
}
```

Topic 2: enriched-metrics

Property	Value
Purpose	Validated, enriched, deduplicated metrics
Partitions	12
Replication Factor	3
Retention	3 days
Producer	Ingestion Cluster

Property	Value
Consumer	ML Cluster

Additional fields after enrichment:

```
{
  "...original fields...",
  "enrichment": {
    "baseline_cpu_avg": 45.2,
    "baseline_cpu_stddev": 12.1,
    "z_score_cpu": 3.49,
    "time_bucket": "weekday_afternoon",
    "is_maintenance_window": false,
    "source_group": "payment-servers"
  }
}
```

Topic 3: anomaly-events

Property	Value
Purpose	Confirmed anomaly detections
Partitions	4 (partitioned by severity: CRITICAL, HIGH, MEDIUM, LOW)
Replication Factor	3
Retention	30 days
Producer	ML Cluster
Consumer	Backend Service, Alert Service, WebSocket Gateway

Message Schema:

```
{
  "anomaly_id": "uuid-v4",
  "source_id": "server-042",
  "timestamp": "2026-09-29T14:30:00.123Z",
  "severity": "CRITICAL",
  "anomaly_type": "cpu_spike",
  "confidence": 0.94,
  "models": {
```

```

    "isolation_forest": {"score": 0.92, "vote": "ANOMALY"},
    "autoencoder": {"score": 0.96, "vote": "ANOMALY"},
    "lstm": {"score": 0.89, "vote": "ANOMALY"}
  },
  "ensemble_decision": "ANOMALY",
  "voting_result": "3/3 unanimous",
  "metric_snapshot": {
    "cpu_percent": 87.5,
    "baseline_avg": 45.2,
    "deviation": "3.49 sigma"
  },
  "recommended_action": "Investigate CPU spike – possible runaway process or
DDoS"
}

```

Partitioning Strategy

Why Partition?

Partitions enable **parallel processing**. Each partition is consumed by exactly one consumer in a consumer group, allowing horizontal scaling.

raw-metrics topic (12 partitions):

```

Partition 0: [server-001, server-013, server-025, ...] → Consumer 1
Partition 1: [server-002, server-014, server-026, ...] → Consumer 2
Partition 2: [server-003, server-015, server-027, ...] → Consumer 3
...
Partition 11: [server-012, server-024, server-036, ...] → Consumer 12

```

- Partition key = source_id → all metrics from one server go to the same partition
- This ensures **ordering per source** (critical for time-series anomaly detection)
- Adding more partitions = adding more consumers = linear throughput scaling

Replication (Fault Tolerance)

```

Broker 1: [P0-Leader, P1-Follower, P2-Follower, P3-Leader, ...]
Broker 2: [P0-Follower, P1-Leader, P2-Follower, P3-Follower, ...]
Broker 3: [P0-Follower, P1-Follower, P2-Leader, P3-Follower, ...]

```

- **Replication Factor = 3**: Each partition has 1 leader + 2 followers
- If Broker 1 dies, followers on Broker 2/3 are elected as new leaders
- **Zero data loss** with acks=all producer config

- The cluster tolerates **1 broker failure** without any downtime
-

Layer 4: Ingestion Cluster

Definition

The **Ingestion Cluster** is a group of stateless microservice instances that consume raw metrics from Kafka, validate them, enrich them with contextual data, and deduplicate them before forwarding to the ML cluster.

Why a Separate Ingestion Layer?

- **Separation of concerns:** Ingestion logic != ML logic
- **Independent scaling:** Can scale ingestion without scaling ML (and vice versa)
- **Data quality gate:** Ensures only clean, valid data reaches the ML models
- **Backpressure handling:** If ML is slow, ingestion buffers in Kafka

Three Core Functions

4.1 Schema Validation

Every incoming metric event is validated against a strict schema:

Incoming Event → Schema Validator → Valid? → Forward
→ Invalid? → Dead Letter Queue

Validation Rules:

Field	Rule	Example Violation
event_id	Must be valid UUID v4	"abc123"
timestamp	ISO 8601, not in future, not > 5min old	"2030-01-01"
cpu_percent	0.0 to 100.0	-5.0 or 150.0

Field	Rule	Example Violation
source_id	Must exist in registry	"unknown-server"
metrics	At least 1 metric present	{}

Invalid events go to a **Dead Letter Queue (DLQ)** for investigation — they are never silently dropped.

4.2 Metadata Enrichment

Raw metrics lack context. The ingestion layer enriches them:

Enrichment	Source	Purpose
Baseline stats (avg, stddev)	Redis cache	Calculate Z-scores
Time bucket	Timestamp	Different baselines for weekday vs weekend
Maintenance window flag	PostgreSQL schedule	Suppress false alarms during maintenance
Source group	Service registry	Group related servers
Historical percentiles	InfluxDB	P50, P95, P99 context

Z-Score Calculation:

$$\text{Z-Score} = (\text{current_value} - \text{baseline_mean}) / \text{baseline_stddev}$$

Example:

Current CPU = 87.5%
Baseline Mean = 45.2%
Baseline StdDev = 12.1%
Z-Score = (87.5 - 45.2) / 12.1 = 3.49

Z > 3.0 → Statistically significant anomaly

4.3 Deduplication (Bloom Filter)

At scale, duplicate events are common (network retries, producer restarts). A **Bloom Filter** efficiently detects duplicates.

What is a Bloom Filter?

A Bloom Filter is a space-efficient probabilistic data structure that tells you:

- “**Definitely NOT seen**” → 100% accurate
- “**Probably seen before**” → Small false positive rate (~1%)

```
event_id → Hash1(id), Hash2(id), Hash3(id) → Check bit array
```

|

All bits set? → Probably duplicate → Drop

Any bit zero? → Definitely new → Process

Why Bloom Filter (Not HashSet)?

Approach	Memory for 10M events	Accuracy
HashSet	~800 MB	100%
Bloom Filter	~12 MB	99% (1% false positive)

At 1M events/sec, memory savings are critical. A 1% false positive rate (dropping 1% of duplicates that aren't actually duplicates) is acceptable.

Implementation: - Bloom Filter size: 10 million bits - Hash functions: 3 (MurmurHash3 variants) - Reset every 5 minutes (time-windowed deduplication)

Layer 5: ML Cluster (Anomaly Detection Engine)

Definition

The **ML Cluster** is an auto-scaling group of Python-based inference servers that run anomaly detection models on enriched metric data. It uses an **ensemble approach** — three different models vote on whether a data point is anomalous.

Why Ensemble (Not a Single Model)?

Single Model Problem	Ensemble Solution
One model has blind spots	Different models catch different anomaly types
High false positive rate	Majority voting reduces false positives
Fails on certain data distributions	Models complement each other
Hard to explain decisions	Voting provides interpretability

The Three Models

5.1 Model A: Isolation Forest

What it is: An unsupervised ML algorithm that isolates anomalies by randomly partitioning data. Anomalies are “few and different” — they get isolated in fewer splits.

How it works:

Normal point: Requires many splits to isolate (deep in the tree)

Anomalous point: Requires few splits to isolate (near the root)

[Split: CPU > 60?]

/ \

[Split: Mem > 70?] ★ Anomaly! (isolated in 1 split)

/ \

[Split...] [Split...]

/ \

Normal Normal (isolated after 4+ splits)

Anomaly Score: Average path length across 100 trees. Short path = anomaly.

Parameter	Value	Reason
n_estimators	100	Number of isolation trees
contamination	0.01	Expected 1% anomaly rate
max_samples	256	Subsample size per tree
Features	CPU, Memory, Disk I/O, Network, Error Rate, Latency	Multi-dimensional

Best at catching: Point anomalies — single data points that are far from normal.

5.2 Model B: Autoencoder (Neural Network)

What it is: A neural network trained to compress (encode) normal data into a small representation and then reconstruct (decode) it. If it fails to reconstruct a data point accurately, that point is anomalous.

Architecture:

Input (8 features) → Encoder → Bottleneck (3 neurons) → Decoder → Output (8 features)

Normal data: Input ≈ Output (low reconstruction error)

Anomaly: Input ≠ Output (high reconstruction error)

Layer Details:

Layer	Neurons	Activation
Input	8	—
Encoder Hidden 1	16	ReLU
Encoder Hidden 2	8	ReLU
Bottleneck	3	ReLU
Decoder Hidden 1	8	ReLU
Decoder Hidden 2	16	ReLU
Output	8	Sigmoid

Anomaly Score: Mean Squared Error (MSE) between input and reconstruction.

```
MSE = (1/n) * Σ(input_i - output_i)2

Threshold = mean(MSE_training) + 3 * std(MSE_training)

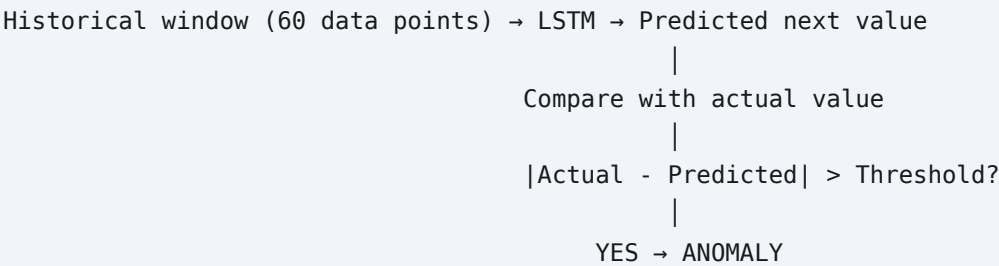
If MSE > Threshold → ANOMALY
```

Best at catching: Contextual anomalies — values that are normal individually but abnormal in combination (e.g., high CPU + low request count = suspicious).

5.3 Model C: LSTM (Long Short-Term Memory)

What it is: A recurrent neural network that learns sequential patterns in time-series data. It predicts the next expected value — if the actual value deviates significantly from the prediction, it’s anomalous.

How it works:

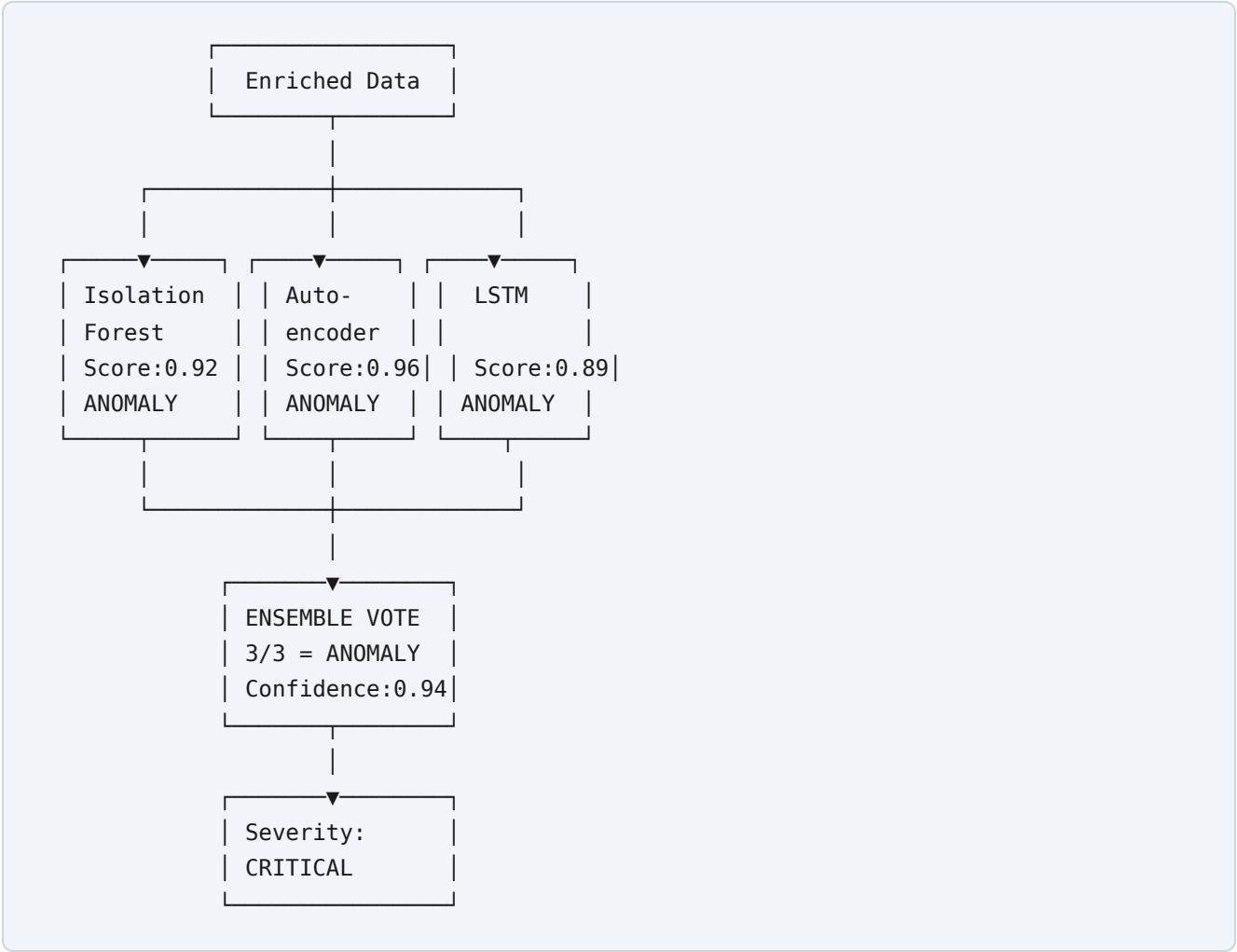


Architecture:

Layer	Details
Input	60 timesteps x 8 features
LSTM Layer 1	64 units, return_sequences=True
Dropout	0.2
LSTM Layer 2	32 units
Dropout	0.2
Dense	8 units (predict next values)

Best at catching: Temporal anomalies — sudden trend breaks, seasonality violations, gradual drifts that other models miss.

Ensemble Voting (Majority Wins)



Voting Rules:

Votes for Anomaly	Decision	Confidence Calculation
3/3 (unanimous)	ANOMALY	avg(scores)
2/3 (majority)	ANOMALY	avg(top 2 scores) * 0.85
1/3	NORMAL	Logged for review
0/3	NORMAL	Not logged

Severity Classification:

Confidence	Z-Score	Severity
> 0.90	> 4.0	CRITICAL
> 0.80	> 3.0	HIGH

Confidence	Z-Score	Severity
> 0.70	> 2.5	MEDIUM
> 0.60	> 2.0	LOW

Auto-Scaling

The ML cluster auto-scales based on Kafka consumer lag:

Kafka Lag	Action	Instances
< 1,000	Scale down	2 (minimum)
1,000 - 10,000	Normal	3
10,000 - 100,000	Scale up	6
> 100,000	Emergency scale	12 (maximum)

Scaling is managed by **Kubernetes Horizontal Pod Autoscaler (HPA)** with custom Kafka lag metrics exported via Prometheus.

Layer 6: Data Layer

6.1 PostgreSQL (Relational Data)

What it stores

Table	Purpose	Example Data
users	Platform users	Engineers, admins
alerts	Alert history	Who was notified, when, response
anomaly_metadata	Anomaly details	Source, type, severity, resolution
sources	Registered data sources	Server registry
alert_rules	Alerting configuration	Thresholds, channels
maintenance_windows	Scheduled downtimes	Suppress false alarms

Why PostgreSQL (Not MySQL or MongoDB)

- ACID compliance for alert audit trails
- JSONB support for flexible metric snapshots
- Advanced indexing (GIN, BRIN) for time-range queries
- Mature, battle-tested at Instagram, Uber, Reddit

6.2 InfluxDB (Time-Series Data)

Definition

InfluxDB is a purpose-built time-series database optimized for high-write throughput and time-range queries on metric data.

What it stores

All metric data points with timestamps — the actual CPU, memory, network values flowing through the system.

Why InfluxDB (Not PostgreSQL for metrics)

Operation	PostgreSQL	InfluxDB
Write 100K metrics/sec	Struggles	Native
Query “avg CPU last 24h”	Slow (scan)	Instant (pre-aggregated)
Auto-delete old data	Manual	Retention policies
Compression	~10x	~50x for time-series
Downsampling	Manual	Continuous queries

Retention Policies

Policy	Duration	Resolution	Purpose
raw	7 days	1 second	Recent detailed data
hourly	90 days	1 hour	Medium-term trends
daily	2 years	1 day	Long-term capacity planning

Data is automatically downsampled: after 7 days, 1-second data is aggregated to hourly averages.

6.3 Redis Cluster

Definition

Redis is an in-memory data store used for caching, real-time pub/sub messaging, and rate limiting. A **Redis Cluster** distributes data across multiple nodes for high availability.

Three Roles in This Architecture

Role 1: Cache (Hot Data)

What’s cached	TTL	Purpose
Baseline stats per source	5 min	Z-score calculation in ingestion
Recent anomalies	1 min	Dashboard quick load
User sessions	30 min	API authentication

What's cached	TTL	Purpose
Source metadata	10 min	Enrichment lookups

Role 2: Pub/Sub (Real-Time Feed)



Redis Pub/Sub delivers anomaly events to the WebSocket Gateway and Alert Service with sub-millisecond latency.

Role 3: Rate Limiting

Sliding window rate limiting using Redis sorted sets:

Key: "ratelimit:user:123"
Score: timestamp
Member: request_id

ZRANGEBYSCORE → count requests in last 60 seconds
Count > limit? → Reject (429)

Redis Cluster Topology

Master 1 (slots 0-5460) → Replica 1a
Master 2 (slots 5461-10922) → Replica 2a
Master 3 (slots 10923-16383) → Replica 3a

- 3 masters + 3 replicas = 6 nodes
- Automatic failover: if a master dies, its replica is promoted
- Data is sharded across masters using hash slots

Layer 7: Observability Stack

Definition

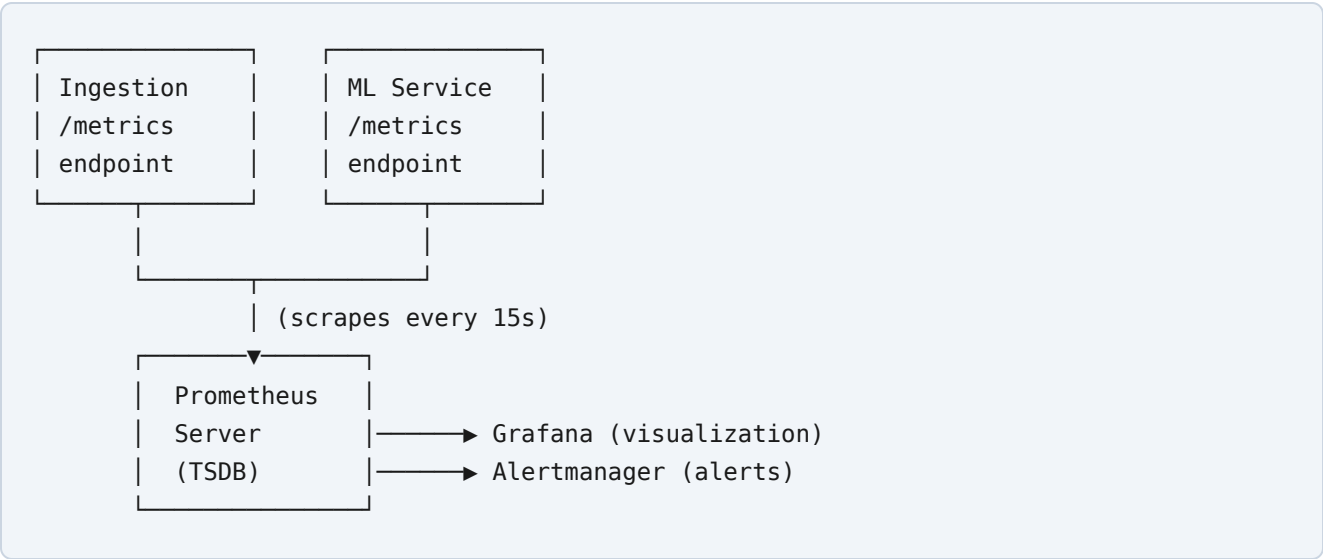
Observability is the ability to understand the internal state of a system by examining its outputs. The three pillars of observability are: **Metrics**, **Traces**, and **Logs**.

7.1 Prometheus (Metrics)

What it monitors

Metric	Type	Example
ingestion_events_total	Counter	Total events processed
ml_inference_duration_seconds	Histogram	Model prediction latency
kafka_consumer_lag	Gauge	Messages waiting to be consumed
anomaly_detections_total	Counter	Anomalies detected (by severity)
websocket_connections_active	Gauge	Current WebSocket connections
alert_notifications_sent	Counter	Alerts sent (by channel)

How Prometheus Works



Prometheus **pulls** metrics from each service's `/metrics` endpoint every 15 seconds.

7.2 Jaeger (Distributed Tracing)

Definition

Distributed tracing tracks a single request as it flows through multiple microservices. Each service adds a “span” to the trace, creating a complete picture of the request’s journey.

Example Trace

```
Trace ID: abc-123-def
|
├── [Ingestion Service] 2ms – Validated and enriched metric
|   |
|   └── [Redis] 0.5ms – Fetched baseline from cache
|
├── [ML Service] 15ms – Ran ensemble prediction
|   |
|   ├── [Isolation Forest] 3ms – Score: 0.92
|   ├── [Autoencoder] 8ms – Score: 0.96
|   └── [LSTM] 4ms – Score: 0.89
|
├── [Backend Service] 3ms – Stored anomaly in DB
|   |
|   ├── [PostgreSQL] 1ms – INSERT anomaly
|   └── [Redis Pub/Sub] 0.3ms – Published to channel
|
└── [Alert Service] 50ms – Sent Slack notification
    |
    └── [Slack API] 45ms – External call

Total: 70ms end-to-end
```

Why Tracing Matters

- Find **bottlenecks**: Which service is slow?
 - Debug **failures**: Where exactly did the request fail?
 - Understand **dependencies**: What calls what?
 - Measure **latency**: End-to-end request time
-

7.3 ELK Stack (Centralized Logging)

Definition

ELK = **Elasticsearch** (search/store) + **Logstash** (ingest/transform) + **Kibana** (visualize)

How It Works

All Services —[Structured JSON logs]→ Logstash → Elasticsearch → Kibana

Log Format (Structured JSON)

```
{
  "timestamp": "2026-09-29T14:30:00.123Z",
  "service": "ml-service",
  "level": "WARN",
  "trace_id": "abc-123-def",
  "message": "High confidence anomaly detected",
  "context": {
    "source_id": "server-042",
    "confidence": 0.94,
    "severity": "CRITICAL",
    "model_votes": "3/3"
  }
}
```

Why Centralized Logging

- With 20+ service instances, you can't SSH into each to read logs
 - Search across all services: "show me all CRITICAL anomalies in the last hour"
 - Correlate logs with traces using trace_id
 - Set up log-based alerts: "alert if ERROR count > 100 in 5 minutes"
-

Complete End-to-End Data Flow

The Journey of a Single Metric Event

Here is exactly what happens when a server sends a CPU metric through the entire system:

Step 1: Data Generation

Server-042 agent collects metrics:

CPU: 87.5%, Memory: 72.3%, Requests: 15420/sec

→ Serializes as JSON

→ Sends to Kafka raw-metrics topic, partition 6 (hash of "server-042")

Step 2: Kafka Receives

Kafka Broker 2 (leader for partition 6):

→ Writes to commit log

→ Replicates to Broker 1 and Broker 3

→ Acknowledges producer (acks=all)

→ Latency: ~2ms

Step 3: Ingestion Consumes

Ingestion Instance 2 (assigned partition 6):

→ Reads event from Kafka

→ Schema validation: PASS (all fields valid)

→ Bloom filter check: NEW (not a duplicate)

→ Enrichment:

- Fetch baseline from Redis: avg=45.2, stddev=12.1

- Calculate Z-score: $(87.5 - 45.2) / 12.1 = 3.49$

- Time bucket: weekday_afternoon

- Maintenance window: false

→ Publishes enriched event to Kafka enriched-metrics topic

→ Latency: ~8ms

Step 4: ML Inference

ML Instance 3 (assigned from enriched-metrics):

→ Receives enriched event

- Feature vector: [87.5, 72.3, 450.2, 120.5, 85.3, 15420, 0.023, 245.7]
- Model A (Isolation Forest): score=0.92 → ANOMALY
- Model B (Autoencoder): reconstruction_error=0.96 → ANOMALY
- Model C (LSTM): prediction_error=0.89 → ANOMALY
- Ensemble: 3/3 votes ANOMALY, confidence=0.94
- Severity: CRITICAL (confidence>0.90, z-score>4.0)
- Publishes to Kafka anomaly-events topic, partition 0 (CRITICAL)
- Latency: ~20ms

Step 5: Backend Processes

Backend Service:

- Consumes from anomaly-events topic
- Stores in PostgreSQL: INSERT INTO anomaly_metadata (...)
- Stores metric snapshot in InfluxDB
- Publishes to Redis channel "anomaly:critical"
- Latency: ~5ms

Step 6: Real-Time Broadcast

Redis Pub/Sub → WebSocket Gateway:

- Receives on channel "anomaly:critical"
- Finds all clients subscribed to "all-anomalies" and "critical" rooms
- Broadcasts JSON event to 847 connected dashboards
- React dashboard updates the live chart and anomaly table
- Latency: ~3ms

Step 7: Alert Fires

Alert Service:

- Consumes from anomaly-events topic
- Checks alert rules: CRITICAL → Slack + Email + SMS
- Deduplication check: no alert for server-042 in last 5 minutes
- Sends Slack: "#alerts-critical: ANOMALY on server-042 – CPU 87.5% (3.49σ)"
- Sends Email: on-call engineer (payments team)
- Sends SMS: +91-XXXXXXXXXX
- Latency: ~100ms (external API calls)

Step 8: Observability Records Everything

Prometheus: Increments anomaly_detections_total{severity="critical"}
Jaeger: Complete trace from ingestion → ML → backend → alert (trace_id: abc-123)
ELK: Structured log entries from every service with trace_id correlation

Total End-to-End Latency

Kafka Ingest: ~2ms

Ingestion: ~8ms

ML Inference: ~20ms

Backend Store: ~5ms

WebSocket Push: ~3ms

Dashboard Update: ~38ms (sub-50ms!)

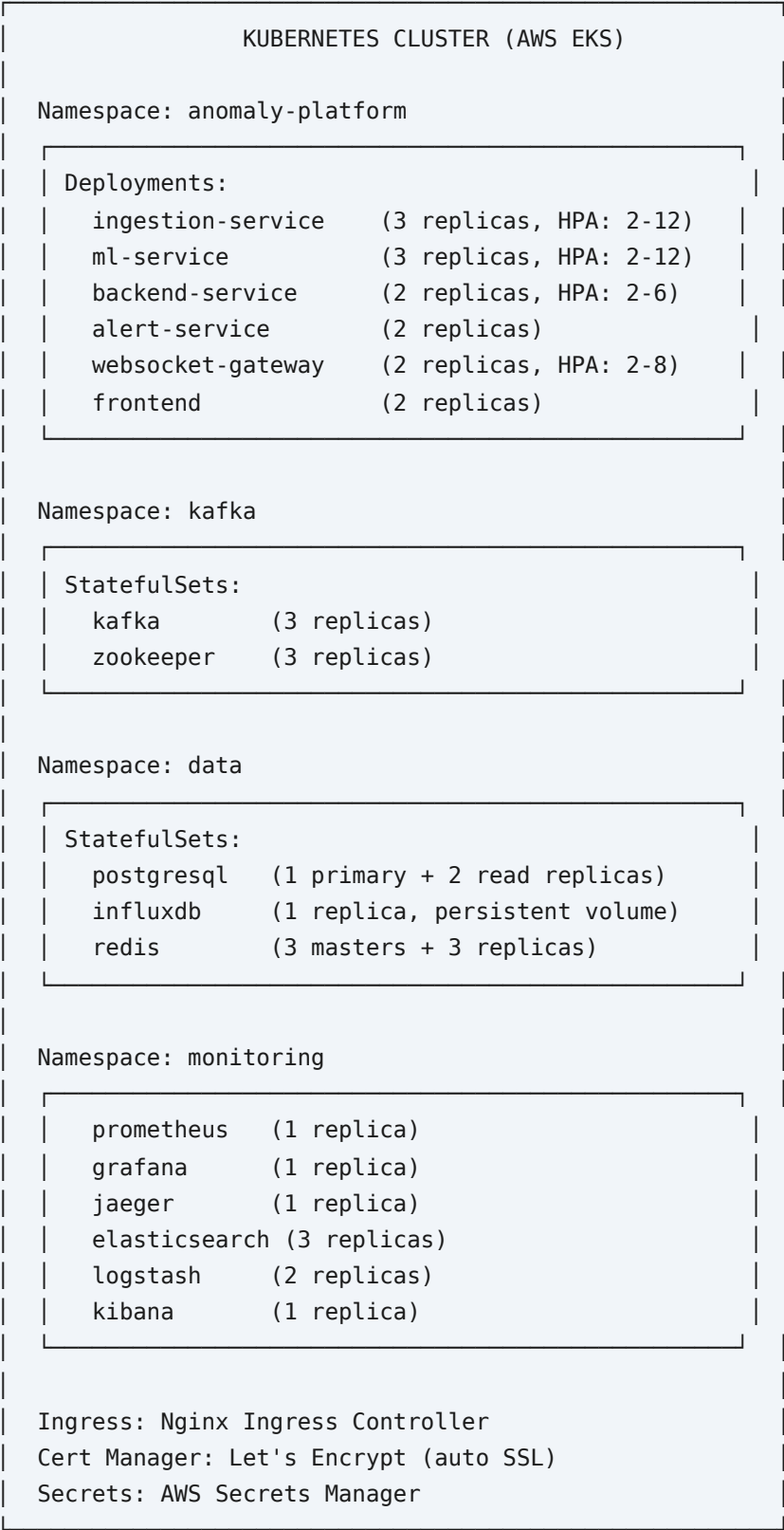
Alert Delivery: ~100ms (external APIs)

Technology Stack Summary

Layer	Technology	Language	Purpose
Load Balancer	Nginx	Config	Traffic distribution, SSL termination
API Gateway	Kong / Spring Cloud Gateway	Java	Auth, rate limiting, routing
WebSocket Gateway	Node.js + ws library	TypeScript	Real-time client connections
Event Streaming	Apache Kafka (3 brokers)	Scala/Java	Message backbone
Ingestion	Spring Boot	Java	Validation, enrichment, dedup
ML Service	FastAPI	Python	Anomaly detection inference
ML Models	scikit-learn, PyTorch	Python	IF, Autoencoder, LSTM
Backend API	Spring Boot	Java	REST APIs, business logic
Alert Service	FastAPI	Python	Multi-channel notifications
Frontend	React + TypeScript	TypeScript	Real-time dashboard
Relational DB	PostgreSQL	SQL	Users, alerts, metadata
Time-Series DB	InfluxDB	InfluxQL	Metric storage and queries
Cache/Pub-Sub	Redis Cluster	—	Caching, real-time feed, rate limiting
Metrics	Prometheus	PromQL	System and business metrics
Tracing	Jaeger	—	Distributed request tracing

Layer	Technology	Language	Purpose
Logging	ELK Stack	—	Centralized log management
Containers	Docker	Dockerfile	Service packaging
Orchestration	Kubernetes	YAML	Deployment, scaling, management
Monitoring Dashboards	Grafana	—	Visualization
Infrastructure	AWS (EKS, EC2, S3)	Terraform	Cloud deployment

Deployment Architecture (Kubernetes)



Interview Talking Points

When discussing this architecture in FAANG interviews, highlight these key decisions:

Scalability Decisions

- 1. **Why Kafka over direct HTTP?** — Decouples producers from consumers, handles backpressure, enables replay
- 2. **Why partition by source_id?** — Ensures ordering per source (critical for LSTM time-series model)
- 3. **Why 3 Kafka brokers?** — Tolerates 1 broker failure with replication factor 3
- 4. **Why auto-scale ML separately?** — ML inference is CPU-heavy; ingestion is I/O-heavy. Different scaling profiles.

Reliability Decisions

- 5. **Why Bloom filter for dedup?** — O(1) lookup, 12MB vs 800MB for HashSet at 10M events
- 6. **Why circuit breakers?** — Prevents cascade failure when ML service is slow
- 7. **Why Dead Letter Queue?** — Invalid events are preserved for debugging, not silently lost
- 8. **Why ensemble voting?** — Reduces false positives compared to any single model

Data Decisions

- 9. **Why InfluxDB + PostgreSQL (not just PostgreSQL)?** — Time-series DB handles 100K writes/sec with 50x compression
- 10. **Why Redis for three roles?** — Already in the stack; cache + pub/sub + rate limiting avoids adding 3 separate systems

Trade-offs You Should Discuss

Decision	Trade-off
Kafka	High throughput but operational complexity
Bloom filter	Memory efficient but 1% false positive rate
Ensemble 3 models	Better accuracy but 3x inference cost

Decision	Trade-off
InfluxDB	Great for metrics but another system to operate
WebSocket	Real-time but connection management overhead
Kubernetes	Powerful but steep learning curve

Conclusion

This Architecture 4 design demonstrates proficiency in:

- **Distributed Systems:** Kafka, partitioning, replication, consumer groups
- **Machine Learning in Production:** Model serving, ensemble, auto-scaling
- **System Design:** Load balancing, API gateway, circuit breakers, rate limiting
- **Data Engineering:** Stream processing, time-series storage, caching strategies
- **DevOps:** Kubernetes, Docker, observability (metrics + tracing + logs)
- **Real-Time Systems:** WebSocket, Pub/Sub, sub-50ms end-to-end latency

This is the kind of architecture that top-tier companies build and the kind of thinking they expect in L4+ system design interviews.

Document prepared by Yash Lohade — September 2026